웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

공개 게시글 조회와 JWT 기반 게시글 관리를 제공하는 서비스의 구현 보고서이다. React 화면, Spring Boot API, SQLite 저장 구조와 인증 및 작성자 권한 검사를 설명한다.

01 서비스 소개와 구현 범위

학번 202212949 이름: 백경환 제출일: 2026.10.28

서비스 이름 우리의 기록

기술 조합과 버전 React 19.3.0 / Vite 8.3.4 / Spring Boot 3.5.16 / Java 17 설정 / JJWT 0.13.0 / SQLite JDBC 3.51.3.0
Node.js 요구 버전: 20.19.0 이상. 설치된 런타임 버전은 실행 시 별도 기록한다.

GitHub 저장소 URL https://github.com/Dosan2883/wsd_assign2

소스 코드 기준 main / 46e49fd

이 서비스는 게시글을 읽고 기록을 남기는 사용자를 위한 게시판이다. 비로그인 사용자도 목록과 상세 내용을 조회할 수 있고, 로그인한 사용자는 글을 작성할 수 있다. 수정과 삭제는 작성자에게만 허용한다. React 는 화면 상태와 API 호출을 담당하고 Spring Boot 는 인증, 권한 검사와 데이터 저장을 담당한다. 개인적인 기술 선택 이유와 제공 템플릿 대비 본인 변경 범위는 작성자가 보완한다.

완료 여부는 코드 구현 기준이며, 실제 동작 검증은 테스트 결과 절에 별도로 기록한다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	공개 GET API 와 목록·상세 화면	
로그인과 JWT 발급	완료	계정 검증과 HS256 JWT 발급	
인증과 내 정보 조회	완료	JWT 검증 후 사용자 조회	
본인 글 작성·수정·삭제	완료	서버 작성자 검사와 CRUD	
만료·로그아웃·오류 표시	완료	401 초기화, 403 유지, 로그아웃	
DB 데이터 유지	완료	SQLite 파일 저장과 자동 테이블 생성	

구조와 변경 내역

브라우저 React → Vite /api 프록시 → Spring Boot(127.0.0.1:8000) → JdbcTemplate → SQLite 파일의 구조이다.
공개 GET /api/posts 와 GET /api/posts/{id}에는 인증이 필요하지 않다.
로그인: POST /api/auth/login → 입력 검사 → Users.verify 의 PBKDF2 검증 → AuthService.issueToken → accessToken, expiresIn, user 반환. 토큰과 사용자 정보는 api.js 의 모듈 메모리 session 에 저장한다.
수정: PUT /api/posts/{id} → Authorization: Bearer 헤더 → JWT 검증 → ID 검사 → 글 존재 확인 → 작성자 비교 → 본문 검사 → SQLite UPDATE → 수정된 글 반환. 토큰이 있으면 공통 request 가 공개 조회를 포함한 API 요청에 헤더를 붙인다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
AuthService.java / issueToken, currentUserId, assertOwner	JWT 발급·검증, 작성자 검사	형식·클레임·사용자 검증 포함	[본인 변경·작성 출처 확인]
frontend-react/src/api.js / login, request, clearSession, canEdit	메모리 인증과 API 통신	401/403 안내와 세션 처리	[본인 변경·작성 출처 확인]
frontend-react/src/main.jsx / App, run, save	로그인·게시판 UI 와 화면 상태	오류별 상태 처리, 작성자 버튼	[본인 변경·작성 출처 확인]

로컬 실행 기록

실행 절차(Windows PowerShell): JDK 17 이상과 Node.js 20.19 이상을 준비한다. backend-spring 에서 JWT_SECRET 을 새로 생성한 64 자리 16 진수로 설정하고 TOKEN_TTL=90 을 지정한다. .\mvnw.cmd package 후 java -jar target/jwt-board-1.0.0.jar 로 실행한다.
frontend-react 에서 잠금 파일이 있으면 npm ci, 없으면 npm install 로 의존성을 설치한 뒤 npm run build, npm run preview 를 실행한다. 기본 DB 는 backend-spring 에서 실행할 때 data/board.sqlite 이다. DB_FILE 로 경로를 변경할 수 있다.

접속 주소: http://127.0.0.1:4173. preview.proxy 가 /api 를 8000 번으로 전달한다. 개발 서버는 5173 번이다.
실제 설치·빌드 로그, 4173 번 API 연결 캡처 및 서버 재시작 결과: [직접 실행 후 작성]

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/BoardController.java 의 login, AuthService.java 의 issueToken

코드 발췌

```
@PostMapping("/auth/login")
public Object login(@RequestBody Map<String, Object> body) {
  var input = fields(body, Map.of("email", 254, "password", 128));
  var user = users.verify(input.get("email"), input.get("password"));
  if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");
  return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);
}
```

동작과 응답 코드 설명

email(최대 254 자)과 password(최대 128 자)만 받는다. fields 는 누락·비문자열·빈 값·초과 길이·추가 필드를 400 BAD_INPUT 으로 거부한다. 이메일은 양끝 공백을 제거하고 비밀번호는 원문을 유지한다. Users.verify 는 PBKDF2WithHmacSHA256(210,000 회) 결과를 비교한다. 계정 불일치는 401 INVALID_CREDENTIALS 이다. 성공하면 200 과 accessToken, expiresIn, user 를 반환한다. issueToken 은 sub=사용자 ID 문자열, iss, aud, exp 를 넣어 HS256 으로 서명한다. 기본 TTL 은 90 초이며 TOKEN_TTL 로 변경한다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-react/src/api.js 의 login, request, authorizationHeaders, clearSession

코드 발췌

```
let session = { accessToken: "", user: null };
export const getSession = () => session;
export function clearSession() {
  session = { accessToken: "", user: null };
}
function authorizationHeaders() {
  if (!session.accessToken) return {};
  return { Authorization: `Bearer ${session.accessToken}` };
}
export async function login(email, password) {
  const result = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
```

```
  session = { accessToken: result.accessToken, user: result.user };
  return result.user;
}
```

동작과 응답 코드 설명

login 은 200 응답의 토큰과 사용자 정보를 session 에 저장한다. localStorage 나 쿠키에는 저장하지 않는다. request 는 토큰이 있으면 Bearer 헤더를 추가하고, 204 는 JSON 파싱 없이 null 로 처리한다. 401 이면 clearSession 후 오류를 전달하며 main.jsx 의 run 은 입력과 수정 상태를 초기화하고 로그인 표시를 갱신한다. 403 은 권한 안내만 표시하고 로그인을 유지한다. 로그아웃은 이 탭의 메모리와 입력 상태를 지우며 새로고침도 메모리 인증 상태를 초기화한다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/AuthService.java 의 currentUserId

코드 발췌

```
Jws<Claims> parsed =
  Jwts.parser()
    .verifyWith(jwtSigningKey)
    .sig()
    .clear()
    .add(Jwts.SIG.HS256)
    .and()
    .requireIssuer(jwtIssuer)
    .requireAudience(jwtAudience)
    .build()
    .parseSignedClaims(parts[1].trim());
```

동작과 응답 코드 설명

Authorization 이 없거나 Bearer 형식이 아니면 401 TOKEN_REQUIRED 이다. JJWT parser 는 서명 키와 HS256 허용 목록, issuer 및 audience 조건으로 서명된 JWT 를 검증한다. 이어서 알고리즘, 문자열 sub, exp 존재, iss/aud 및 양의 정수 sub 를 검사하고 Users.find 로 사용자의 존재를 확인한다. 만료 예외는 401 TOKEN_EXPIRED, 서명·형식·클레임 오류는 401 INVALID_TOKEN 이다. 성공하면 사용자 ID 를 반환하여 /me 나 변경 API 가 사용한다. Errors.api 는 401 에 WWW-Authenticate: Bearer 를 추가하고 code 와 message 를 JSON 으로 반환한다.

3 4 작성자 권한 검사

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/BoardController.java 의 update, delete, create 및 AuthService.java 의 assertOwner

코드 발췌

```
@PutMapping("/posts/{id}")
public Object update(
   @PathVariable String id, @RequestBody Map<String, Object> body, HttpServletRequest req) {
  long user = auth.currentUserId(req), pid = postId(id);
  var p = existing(pid);
  auth.assertOwner(user, p.authorId());
  var input = fields(body, Map.of("title", 100, "content", 5000));
  return store.update(pid, input.get("title"), input.get("content"));
}
```

```
 public void assertOwner(long userId, long authorId) {
  if (userId != authorId) {
    throw new ApiError(403, "FORBIDDEN", "게시글 작성자만 수정하거나 삭제할 수 있습니다.");
  }
 }
```

동작과 응답 코드 설명

update/delete 는 토큰에서 얻은 사용자 ID 와 DB 게시글의 authorId 를 assertOwner 로 비교한다. 불일치는 403 FORBIDDEN 이다. 인증 실패 401 은 권한 검사보다 먼저 발생한다. create 는 검증한 사용자 ID 를 작성자로 전달하며, title/content 외 authorId 또는 userId 필드는 400 BAD_INPUT 으로 거부한다. 정상 수정은 200, 삭제는 본문 없는 204, 등록은 Location 헤더와 201 을 반환한다. 정상 토큰으로 없는 글을 변경하면 404 NOT_FOUND 이다. 정상 JSON 요청의 처리 순서는 인증 → ID → 존재 → 소유권 → 입력 검증이다.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: windows 11/ / Chrome / .₩mvnd.cmd springboot:run, npm run dev

로컬 배포 확인: [새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173 번 접속 및 API 연결 증거]

테스트 일시 및 코드 버전: 2026.10.28

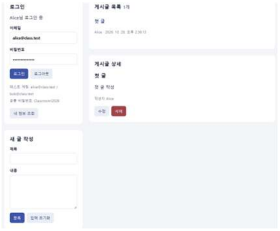
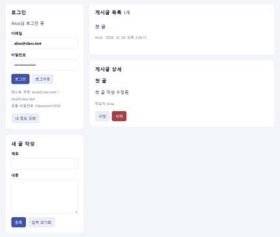
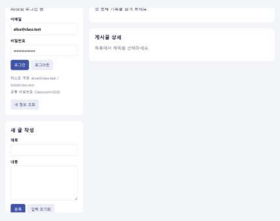
토큰 TTL: 기본 설정 90 초. 실제 일반·만료 테스트에 사용한 값: 90

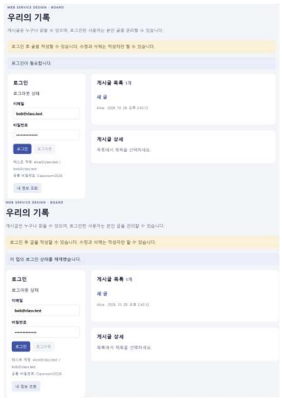
05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	토큰 없이 목록 및 존재하는 글 상세 조회	200, 정상 공개 조회	GET 200 OK	PASS	

T02	Alice 계정으로 로그인	200, JWT 와 공개 사용자 정보, 화면 로그인 표시	200 OK, {
			"accessToken": "****(직접 가림)",
			"expiresIn": 90,
			"user": {
			"id": 1,
			"name": "Alice",
			"email": "alice@class.test"
			}
			}
				PASS	

T03	잘못된 비밀번호 및 없는 이메일로 각각 로그인	401 INVALID_CREDENTIALS, 토큰 미발급	401 Unauthorized, {"code":"INVALID_CREDENTIALS","message":"이메일 또는 비밀번호를 확인하세요."}	PASS	

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T04	토큰 없이 /me 와 글 등록 요청	각각 401 TOKEN_REQUIRED	등록 버튼 X	미실행	[실행 후 첨부]
T05	Alice 토큰으로 글 작성 후 /me 조회	201, authorId 1, /me 는 Alice	201 Created, { "id": 1, "title": "첫 글", "content": "첫 글 작성", "authorId": 1, "authorName": "Alice", "createdAt": "2026-10-28T05:36:13.015723500Z", "updatedAt": "2026-10-28T05:36:13.015723500Z" }	PASS	
T06	Alice 토큰으로 본인 글 수정·삭제	200 및 204, 결과가 DB 에 반영	GET 200 OK, [{ "id": 1, "title": "첫 글", "content": "첫 글 작성 수정중", "authorId": 1, "authorName": "Alice", "createdAt": "2026-10-28T05:36:13.015723500Z", "updatedAt": "2026-10-28T05:38:39.028374500Z" }] GET 200 OK, []	PASS	 
T07	Alice 글을 Bob 의 정상 토큰으로 수정	403 FORBIDDEN, 원문 유지	수정 버튼 X	미실행	[실행 후 첨부]
T08	Alice 글을 Bob 의 정상 토큰으로 삭제	403 FORBIDDEN, 글 유지	삭제 버튼 X	미실행	[실행 후 첨부]

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T09	유효 토큰의 서명 부분 첫 글자를 다른 문자로 바꿔 /me 및 변경 요청	401 INVALID_TOKEN, 데이터 변경 없음	미측정	미실행	[실행 후 첨부]
T10	짧은 TTL 로 발급하고 실제 만료 뒤 /me 및 변경 요청	401 TOKEN_EXPIRED, 화면 로그인 해제와 안내	미측정	미실행	[실행 후 첨부]
T11	로그인 후 로그아웃, 다음 /me 요청	헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED	401 Unauthorized, {"code":"TOKEN_REQUIRED","message":"로그인이 필요합니다."}	PASS	
T12	Bob 로그인 상태로 글 등록 요청에 authorId:1 또는 userId:1 추가	400 BAD_INPUT, 위조 글 미생성	미측정	미실행	[실행 후 첨부]
T13	본인 토큰으로 빈 제목·공백 본문·길이 초과 입력을 각각 전송	400 BAD_INPUT, DB 변경 없음	미측정	미실행	[실행 후 첨부]
T14	없는 양의 정수 ID 의 조회·수정·삭제 및 잘못된 ID 조회	없는 글 404, 잘못된 ID 400; 수정·삭제에는 정상 토큰 사용	미측정	미실행	[실행 후 첨부]
T15	글을 만든 뒤 서버 종료·재실행	기존 글과 작성자·내용 유지	미측정	미실행	[실행 후 첨부]
T16	로그인 후 페이지 새로고침	공개 글 유지, 메모리 로그인 상태 해제	미측정	미실행	[실행 후 첨부]
T17	토큰 없는 수정·삭제 요청과 Bob 계정의 정상 작성·본인 수정·삭제	무토큰은 401, Bob 본인 작업은 성공	미측정	미실행	[실행 후 첨부]

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	미측정	[실행 후 첨부]
새 글 등록	**201 및 Location**	미측정	[실행 후 첨부]
본인 글 삭제	**204 및 빈 본문**	미측정	[실행 후 첨부]
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	미측정	[실행 후 첨부]
잘못된 계정	**401 INVALID_CREDENTIALS**	미측정	[실행 후 첨부]
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	미측정	[실행 후 첨부]
변조·클레임 오류	**401 INVALID_TOKEN**	미측정	[실행 후 첨부]
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	미측정	[실행 후 첨부]
다른 작성자의 변경	**403 FORBIDDEN**	미측정	[실행 후 첨부]
없는 글	**404 NOT_FOUND**	미측정	[실행 후 첨부]

코드상 Errors.api 는 401 응답에 WWW-Authenticate: Bearer 를 넣는다. 실제 헤더 확인 결과와 증거: [작성]

코드상 401 은 session 을 지우고 입력·수정 상태를 초기화한다. 403 은 로그인 상태를 유지하고 작성자만 수정·삭제할 수 있다는 안내를 표시한다. 실제 화면 확인: [작성]

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

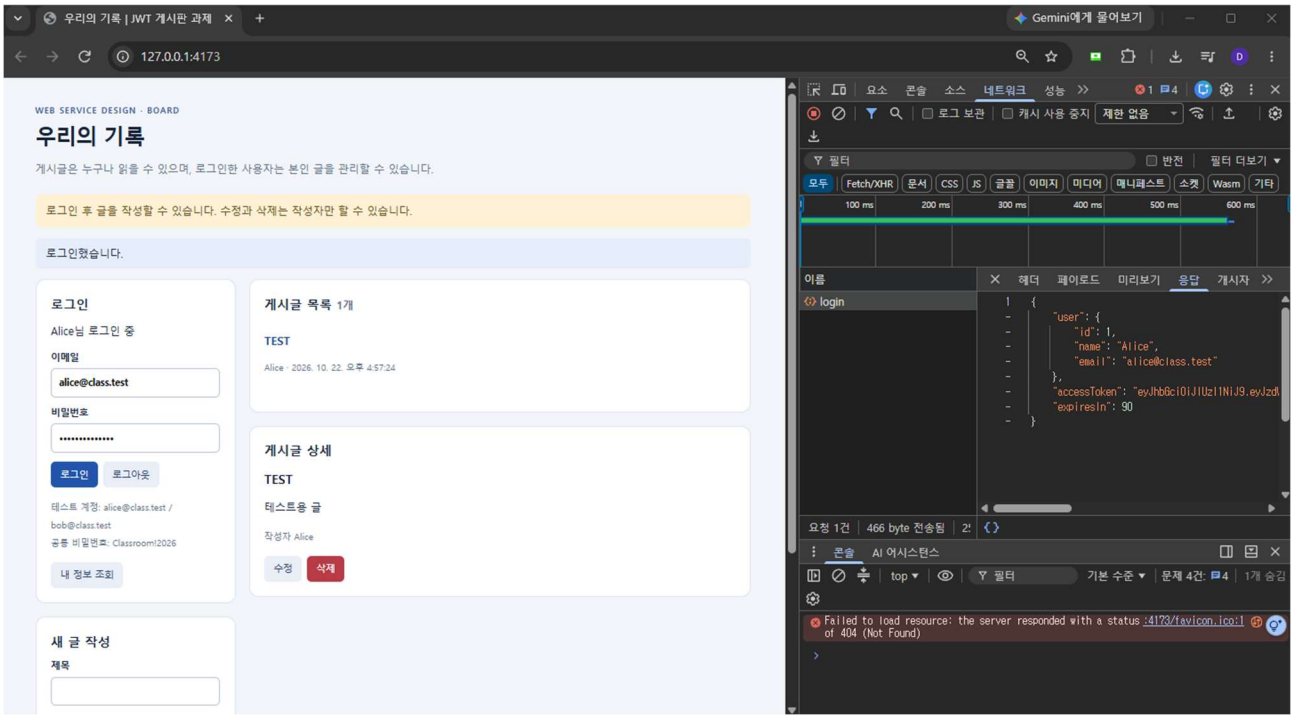
웹서비스설계 04 계속

테스트 증거

캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

로그인과 토큰 전송

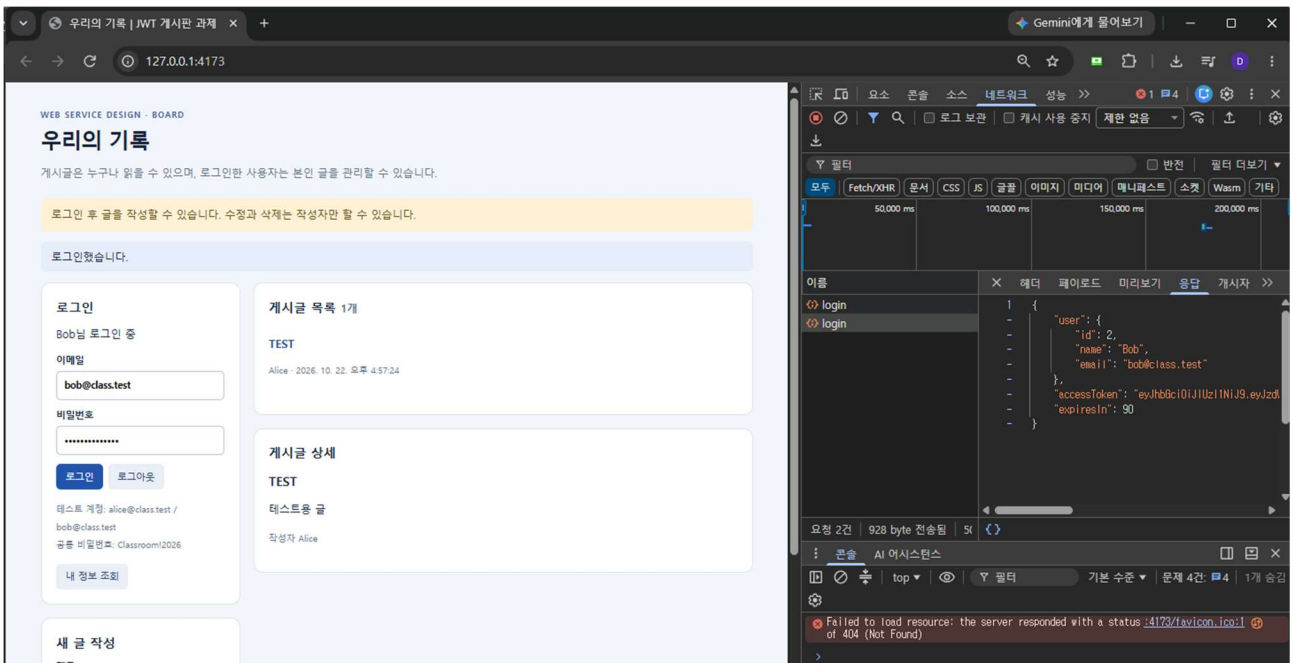
[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]



다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

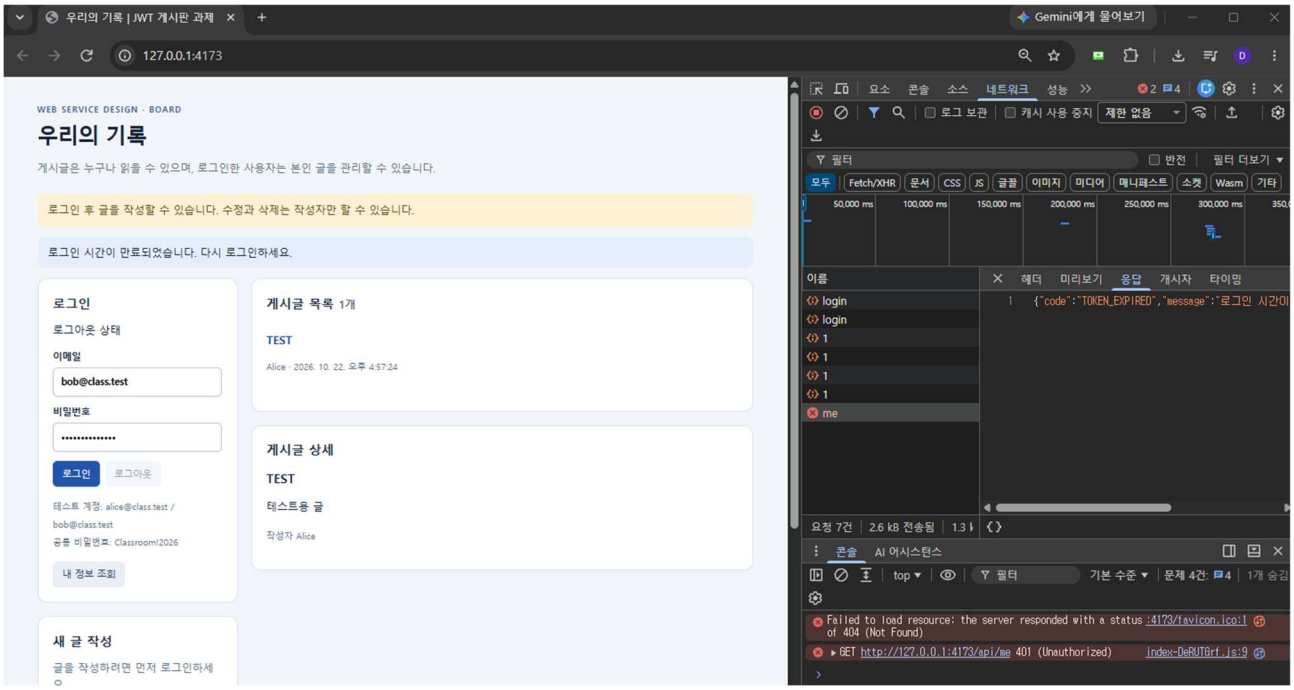
[화면 캡처 또는 요청·응답 삽입]



토큰 변조와 만료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]

[화면 캡처 또는 요청·응답 삽입]



자동 검사와 로컬 배포 증거

[자동 검사 결과를 첨부했다면 실행 명령과 의미를 설명]

[빌드 성공 로그 / 4173 번 API 연결 / 재시작 유지의 화면 또는 파일 경로]

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: 사용

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
ChatGPT Codex 6.0 Astra Medium	설계 / 보고서 작성	보고서 02, 03	그대로 반영
ChatGPT Codex 6.0 Luna Extra High	코드 작성	appservice.java, app.js, main.jsx	그대로 반영

대표 사례

도구 / 날짜: ChatGPT 6.0 Astra Medium

요청한 내용: 코드에서 제공하는 부분과 TODO 에서 코드를 어떻게 만들지 계획

AI 가 제안한 내용: TODO 를 만족하기 위해 자세한 계획

실제 반영 위치: appservice.java, app.js

내가 확인하고 수정한 내용: 수정 X, AI 가 계획한 부분이 타당하다고 생각함

검증 방법과 결과: 코드를 작성하고 잘 수행되는 지 확인했고 잘 수행됨

도구 / 날짜: ChatGPT 6.0 Luna Extra High

요청한 내용: 계획한 부분에 맞춰 코드를 작성하게 함

AI 가 제안한 내용: 제안하지 않았음

실제 반영 위치: appservice.java, app.js, main.jsx

내가 확인하고 수정한 내용: 수정 X, 바뀐 부분에서 해당 코드가 프롬프트에 맞게 작성되었음을 확인했기 때문이다.

검증 방법과 결과: 코드를 작성하고 잘 수행되는 지 확인했고 잘 수행됨

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
[수업 자료 포함]	[파일/함수]	[작성]

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

본문의 authorId 를 신뢰하면 Bob 도 Alice 의 ID 를 보내 다른 사람 명의로 글을 만들 수 있다. create 는 auth.currentUserId 가 검증한 sub 에서 사용자 ID 를 얻어 store.create 에 전달한다. fields 는 title 과 content 만 허용하므로 authorId 나 userId 를 추가하면 400 BAD_INPUT 이다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼 표시 조건은 브라우저에서 우회할 수 있고 API 에 직접 요청할 수도 있다. canEdit 은 사용자 편의를 위한 표시 조건이며, 서버의 update/delete 가 auth.assertOwner(user, p.authorId())를 호출해 실제 소유권을 검사한다. Bob 의 정상 토큰으로 Alice 글을 변경하면 403 FORBIDDEN 이다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰 내용을 읽는 것이며 서명이 유효함을 보장하지 않는다. currentUserId 는 verifyWith 와 parseSignedClaims 로 서명을 검증하고 HS256, iss, aud, 만료를 검사한다. 추가로 sub 의 문자열·양의 정수 형식, exp 존재, 실제 사용자 존재를 확인한 뒤 사용자 ID 를 반환한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 현재 탭의 session 과 입력 상태를 지우고, 새로고침은 모듈 메모리를 재생성한다. 서버에서 이미 발급한 토큰을 폐기하는 기능은 없다. 복사해 둔 토큰은 서명 키 등이 유지되는 동안 만료 전까지 사용할 수 있다. 만료는 exp 에 따라 서버가 거부하며, 프론트는 다음 보호 요청에서 401 을 받으면 로그인을 해제한다. 자동 만료 타이머는 없다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

로그인 할 때 500 오류가 나오면서 해당 원인을 찾느라 힘들었고 오류코드를 분석하면서 환경변수를 설정하지 않았다는 원인을 확인하였고 환경변수 설정을 하고 다시 로그인하여 로그인이 되는 것을 확인했다.

현재 구조의 한계와 개선 후보: 로그아웃 후 기존 토큰의 즉시 폐기가 없고, 사용자 계정은 고정된 두 계정으로 메모리에 구성된다. 필요하면 서버 측 토큰 폐기 정책과 사용자 저장·가입 기능을 추가할 수 있다.

실제 남은 오류와 본인의 개선 계획: [작성]

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출